

Algorithmen und Programmierung

Informatik · Klasse 7 · Arbeitsheft

Inhaltsverzeichnis

06 - Was ist ein Algorithmus?	3
Rückblick	3
Ein Ablauf mit klaren Schritten	3
Beispiel: Kakao zubereiten	3
Aufgabe 1 - Genau genug?	3
Aufgabe 2 - Algorithmus im Alltag	3
Aufgabe 3 - Reihenfolge	3
Aufgabe 4 - Testen	3
Merksatz	4
Ausblick	4
07 - Wie kann man Algorithmen darstellen?	5
Rückblick	5
Textform	5
Ablaufdiagramm	5
Sequenz	5
Entscheidung	5
Wiederholung	5
Aufgabe 1	5
Aufgabe 2	6
Aufgabe 3	6
Aufgabe 4 - Fehler finden	6
Das Wichtigste	6
Ausblick	6
08 - Wie bringen wir einem Computer einen Ablauf bei?	7
Rückblick	7
Scratch als Werkzeug	7
Das erste Programm	7
EVAS wiedererkennen	7
Aufgabe 1	7
Aufgabe 2	7
Aufgabe 3 - Vorhersagen	7
Aufgabe 4 - Debugging	8
Aufgabe 5 - Eigener Mini-Ablauf	8
Merksatz	8
Ausblick	8
09 - Wie reagiert ein Programm auf Eingaben?	9
Rückblick	9
Ereignisse	9
Tastatursteuerung	9
Fragen und Antworten	9
Aufgabe 1 - Steuerung	9
Aufgabe 2 - Begrüßung	9
Aufgabe 3 - EVAS	9
Aufgabe 4 - Testen	9
Aufgabe 5 - Mini-Projekt	9
Ausblick	10
10 - Wie trifft ein Programm Entscheidungen?	11
Rückblick	11
Bedingungen	11
Wenn ... dann	11

Wenn ... dann ... sonst	11
Vergleichsoperatoren	11
Aufgabe 1 – Passwort	11
Aufgabe 2 – Zahl prüfen	11
Aufgabe 3 – EVAS	11
Aufgabe 4 – Fehler finden	12
Aufgabe 5 – Mini-Quiz	12
Merksatz	12
Ausblick	12
11 – Wie wiederholt ein Programm Abläufe?	13
Rückblick	13
Wiederholungen	13
Warum Schleifen?	13
Aufgabe 1	13
Aufgabe 2	13
Aufgabe 3	13
Aufgabe 4	13
Aufgabe 5 – Testen	13
Ausblick	14
12 – Wie merkt sich ein Programm Werte?	15
Rückblick	15
Variablen	15
Wert setzen und verändern	15
Verbindung zum Speicher	15
Aufgabe 1 – Punktestand	15
Aufgabe 2 – Zurücksetzen	15
Aufgabe 3 – Kleines Quiz	15
Aufgabe 4 – Vorhersagen	15
Aufgabe 5 – Transfer	15
Das Wichtigste	16
Ausblick	16
13 – Mein erstes Scratch-Projekt	17
Auftrag	17
Schritt 1 – Idee	17
Schritt 2 – Algorithmus	17
Schritt 3 – Umsetzung	17
Schritt 4 – Testen	17
Schritt 5 – Verbessern	17
Schritt 6 – Erklären	17
Reflexion	17
Ausblick	18
14 – Kompetenzcheck	19
Teil A – Binärzahlen	19
Teil B – Digitale Information	19
Teil C – EVAS	19
Teil D – Algorithmen	19
Teil E – Scratch	19
Auswertung	19

06 - Was ist ein Algorithmus?

Rückblick

Beim EVAS-Prinzip haben wir gesehen, dass ein System Eingaben verarbeitet und Ergebnisse ausgibt.

Leitfrage: Woher weiß ein Computer, wie er eine Eingabe verarbeiten soll?

Ein Ablauf mit klaren Schritten

Ein **Algorithmus** ist eine eindeutige Folge von Anweisungen, mit der eine Aufgabe gelöst wird.

Ein guter Algorithmus ist:

- **eindeutig** – jeder Schritt ist verständlich,
- **ausführbar** – die Schritte können tatsächlich durchgeführt werden,
- **endlich** – der Ablauf kommt zu einem Ende,
- **geordnet** – die Reihenfolge ist sinnvoll.

Beispiel: Kakao zubereiten

Ungeeignet:

Mach einen Kakao.

Besser:

1. Stelle eine Tasse bereit.
2. Gib zwei Löffel Kakaopulver hinein.
3. Gieße warme Milch in die Tasse.
4. Rühre um.

Aufgabe 1 - Genau genug?

Erkläre, warum die Anweisung „Gehe zur Tür“ für einen Roboter zu ungenau sein kann.

Aufgabe 2 - Algorithmus im Alltag

Schreibe einen Algorithmus zum Zähneputzen mit mindestens sechs Schritten.

Aufgabe 3 - Reihenfolge

Bringe in eine sinnvolle Reihenfolge:

- Ergebnis anzeigen
- zwei Zahlen eingeben
- Zahlen addieren
- Programm starten

Aufgabe 4 - Testen

Tausche deinen Zähneputz-Algorithmus mit einem Partner. Führt nur genau das aus, was dort steht. Markiert unklare Stellen.

Merksatz

Ein Computer errät nichts. Ein Algorithmus muss so eindeutig sein, dass die einzelnen Schritte ohne zusätzliche Erklärungen ausgeführt werden können.

Ausblick

Wie kann man Algorithmen so darstellen, dass man ihre Struktur schnell erkennt?

07 - Wie kann man Algorithmen darstellen?

Rückblick

Ein Algorithmus ist eine eindeutige, ausführbare und endliche Folge von Schritten.

Leitfrage: Wie kann man Abläufe so darstellen, dass man ihre Struktur schnell erkennt?

Textform

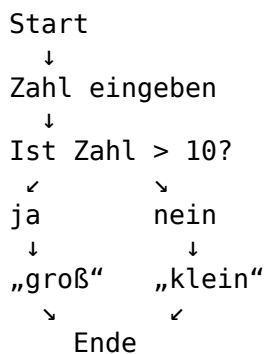
Der einfachste Weg ist eine nummerierte Schrittfolge.

Ablaufdiagramm

Abläufe lassen sich auch grafisch darstellen. Für den Einstieg verwenden wir drei Arten von Elementen:

- **Start/Ende**
- **Anweisung**
- **Entscheidung**

Beispiel:



Sequenz

Eine **Sequenz** bedeutet: Anweisungen werden nacheinander ausgeführt.

Entscheidung

Bei einer Entscheidung hängt der weitere Ablauf von einer Bedingung ab.

Wiederholung

Manche Schritte sollen mehrfach ausgeführt werden. Eine solche Wiederholung nennt man **Schleife**.

Aufgabe 1

Zeichne ein Ablaufdiagramm für:

1. Programm starten.
2. Zahl eingeben.
3. Zahl verdoppeln.
4. Ergebnis ausgeben.

5. Programm beenden.

Aufgabe 2

Erweitere das Diagramm: Wenn das Ergebnis größer als 20 ist, soll zusätzlich „großes Ergebnis“ ausgegeben werden.

Aufgabe 3

Nenne jeweils ein Alltagsbeispiel für Sequenz, Entscheidung und Wiederholung.

Aufgabe 4 - Fehler finden

Ein Algorithmus soll fünfmal „Hallo“ ausgeben. Jemand schreibt fünf identische Ausgabeanweisungen. Welche bessere Struktur könnte verwendet werden?

Das Wichtigste

- Sequenz: Schritte nacheinander
- Entscheidung: Ablauf hängt von einer Bedingung ab
- Wiederholung: Schritte werden mehrfach ausgeführt

Ausblick

Können wir diese Strukturen direkt in einem Programm zusammensetzen, ohne zuerst eine komplizierte Programmiersprache zu lernen?

08 - Wie bringen wir einem Computer einen Ablauf bei?

Rückblick

Algorithmen bestehen aus eindeutigen Schritten. Jetzt setzen wir solche Schritte als Programm um.

Leitfrage: Wie können wir einen Algorithmus programmieren, ohne zuerst komplizierte Befehle tippen zu müssen?

Scratch als Werkzeug

Scratch ist eine blockbasierte Programmiersprache. Befehle werden als Blöcke zusammengesetzt.

Wichtige Bereiche:

- **Bühne** – dort läuft das Programm.
- **Figuren** – Objekte, die Befehle ausführen.
- **Blockpalette** – enthält Befehle.
- **Skriptbereich** – hier werden Blöcke verbunden.

Das erste Programm

Ziel:

Wenn die grüne Fahne angeklickt wird, soll die Figur „Hallo!“ sagen.

Dazu brauchen wir:

1. ein **Ereignis** als Start,
2. eine **Anweisung**.

wenn grüne Flagge angeklickt
sage „Hallo!“ für 2 Sekunden

EVAS wiedererkennen

- Eingabe: Klick auf die grüne Fahne
- Verarbeitung: Scratch führt die Blöcke aus
- Ausgabe: Figur zeigt „Hallo!“

Aufgabe 1

Erstelle das Hallo-Programm und teste es mehrfach.

Aufgabe 2

Erweitere es:

1. Figur sagt „Hallo!“
2. Figur geht 50 Schritte.
3. Figur dreht sich um 90 Grad.
4. Figur sagt „Fertig!“

Aufgabe 3 - Vorhersagen

Vertausche zwei Blöcke. Was verändert sich? Notiere zuerst deine Vermutung, teste danach.

Aufgabe 4 - Debugging

Ein Programm startet nicht. Nenne mindestens zwei mögliche Ursachen.

Aufgabe 5 - Eigener Mini-Ablauf

Erstelle einen Ablauf mit mindestens fünf Blöcken. Schreibe vorher den Algorithmus in Textform auf.

Merksatz

Ein Programm ist eine Umsetzung von Algorithmen in einer Programmiersprache.

Ausblick

Wie kann ein Programm auf eine Eingabe der Benutzerin oder des Benutzers reagieren?

09 - Wie reagiert ein Programm auf Eingaben?

Rückblick

Unser erstes Scratch-Programm startete mit einem Ereignis.

Leitfrage: Wie kann ein Programm auf Tastatur, Maus oder Fragen reagieren?

Ereignisse

Ein Ereignis kann einen Ablauf starten, zum Beispiel:

- grüne Fahne angeklickt
- Taste gedrückt
- Figur angeklickt

Tastatursteuerung

Beispiel:

wenn Taste Pfeil rechts gedrückt
gehe 10 Schritte

Damit wird eine Benutzereingabe direkt verarbeitet.

Fragen und Antworten

Scratch kann auch Text abfragen:

frage „Wie heißt du?“ und warte
sage Antwort

Die Antwort wird gespeichert und kann danach verwendet werden.

Aufgabe 1 - Steuerung

Programmiere eine Figur so, dass sie sich mit vier Pfeiltasten bewegt.

Aufgabe 2 - Begrüßung

Frage nach dem Namen und gib anschließend eine persönliche Begrüßung aus.

Aufgabe 3 - EVAS

Beschreibe dein Programm aus Aufgabe 2 nach EVAS.

Aufgabe 4 - Testen

Teste ungewöhnliche Eingaben: leerer Name, sehr langer Text, Zahl statt Name. Was passiert?

Aufgabe 5 - Mini-Projekt

Erstelle eine kleine interaktive Szene mit mindestens zwei unterschiedlichen Eingaben und zwei sichtbaren Ausgaben.

Ausblick

Bisher führen Programme auf eine Eingabe immer denselben Ablauf aus. Wie kann ein Programm unterschiedliche Wege wählen?

10 - Wie trifft ein Programm Entscheidungen?

Rückblick

Scratch kann auf Ereignisse reagieren. Bisher folgte danach immer derselbe Ablauf.

Leitfrage: Wie kann ein Programm abhängig von einer Bedingung unterschiedliche Wege wählen?

Bedingungen

Eine **Bedingung** kann wahr oder falsch sein, zum Beispiel:

- Taste Leertaste gedrückt?
- Figur berührt Farbe Rot?
- Antwort = 7?

Wenn ... dann

```
wenn <Bedingung> dann  
  Anweisung
```

Die Anweisung wird nur ausgeführt, wenn die Bedingung wahr ist.

Wenn ... dann ... sonst

```
wenn <Antwort = 7> dann  
  sage „richtig“  
sonst  
  sage „noch einmal versuchen“
```

Damit entstehen zwei mögliche Wege.

Vergleichsoperatoren

Scratch kann Werte vergleichen:

- < kleiner als
- > größer als
- = gleich

Aufgabe 1 - Passwort

Frage nach einem vereinbarten Kennwort. Gib „Zugang erlaubt“ oder „Zugang abgelehnt“ aus.

Aufgabe 2 - Zahl prüfen

Frage nach einer Zahl. Wenn sie größer als 10 ist, sage „größer als 10“, sonst „höchstens 10“.

Aufgabe 3 - EVAS

Beschreibe Aufgabe 2 nach Eingabe, Verarbeitung und Ausgabe.

Aufgabe 4 - Fehler finden

Ein Quiz sagt bei jeder Antwort „richtig“. Nenne mögliche Ursachen und beschreibe, wie du den Fehler testest.

Aufgabe 5 - Mini-Quiz

Erstelle eine Frage mit mindestens zwei möglichen Ausgaben. Teste eine richtige und mindestens zwei falsche Eingaben.

Merksatz

Eine Bedingung entscheidet nicht selbst. Das Programm prüft, ob eine Aussage wahr oder falsch ist, und folgt dem passenden Ablauf.

Ausblick

Wie vermeiden wir es, denselben Block zehnmal hintereinander einzubauen?

11 - Wie wiederholt ein Programm Abläufe?

Rückblick

Mit Bedingungen können Programme unterschiedliche Wege wählen.

Leitfrage: Wie führen wir denselben Ablauf mehrfach aus, ohne ihn mehrfach zu programmieren?

Wiederholungen

Eine **Schleife** wiederholt Anweisungen.

Feste Anzahl

```
wiederhole 10 mal  
  gehe 10 Schritte
```

Fortlaufend

```
wiederhole fortlaufend  
  gehe 5 Schritte  
  pralle vom Rand ab
```

Bis etwas geschieht

```
wiederhole bis <berührt Ziel?>  
  gehe 5 Schritte
```

Warum Schleifen?

Schleifen machen Programme kürzer, verständlicher und leichter veränderbar.

Aufgabe 1

Lass eine Figur mit einer Schleife ein Quadrat laufen.

Aufgabe 2

Ändere das Quadrat zu einem regelmäßigen Dreieck. Welche Werte müssen angepasst werden?

Aufgabe 3

Erstelle eine Animation, die fortlaufend läuft und am Rand nicht verschwindet.

Aufgabe 4

Vergleiche zwei Lösungen: zehnmal derselbe Bewegungsblock versus eine Wiederholung. Nenne zwei Vorteile der Schleife.

Aufgabe 5 - Testen

Was passiert, wenn eine Endlosschleife keine Möglichkeit zum Stoppen hat? Wie kannst du dein Programm trotzdem kontrollieren?

Ausblick

Wie kann sich ein Programm Werte merken, die sich während des Ablaufs verändern?

12 - Wie merkt sich ein Programm Werte?

Rückblick

Schleifen wiederholen Abläufe. Manche Programme müssen dabei zählen oder Zwischenergebnisse merken.

Leitfrage: Wo speichert ein Programm Werte, die sich während des Ablaufs verändern?

Variablen

Eine **Variable** ist ein benannter Speicherplatz für einen Wert.

Beispiele:

- Punkte
- Leben
- Zeit
- Anzahl richtiger Antworten

Wert setzen und verändern

setze Punkte auf 0

ändere Punkte um 1

Zu Beginn eines Spiels wird der Punktestand oft auf 0 gesetzt. Bei einem Treffer wird er erhöht.

Verbindung zum Speicher

Eine Variable macht die Idee des Speicherns sichtbar: Das Programm kann einen Wert ablegen, lesen und verändern.

Aufgabe 1 - Punktestand

Erstelle eine Variable Punkte. Beim Klick auf eine Figur soll der Punktestand um 1 steigen.

Aufgabe 2 - Zurücksetzen

Sorge dafür, dass beim Start mit der grünen Fahne immer bei 0 begonnen wird.

Aufgabe 3 - Kleines Quiz

Erstelle drei Fragen. Für jede richtige Antwort gibt es einen Punkt. Am Ende wird der Punktestand ausgegeben.

Aufgabe 4 - Vorhersagen

Was passiert, wenn du das Zurücksetzen der Variable am Programmstart vergisst?

Aufgabe 5 - Transfer

Nenne drei weitere Anwendungen, bei denen ein Programm Werte speichern und verändern muss.

Das Wichtigste

- Variablen speichern Werte unter einem Namen.
- Werte können gelesen, gesetzt und verändert werden.
- Variablen verbinden Speicherung mit Algorithmen und Programmen.

Ausblick

Jetzt kennen wir Sequenzen, Bedingungen, Schleifen und Variablen. Können wir daraus selbstständig ein kleines Programm entwickeln?

13 - Mein erstes Scratch-Projekt

Auftrag

Entwickle ein kleines Scratch-Projekt, in dem du mehrere bisher gelernte Bausteine kombinierst.

Dein Projekt soll mindestens enthalten:

- ein Ereignis,
- eine Eingabe,
- eine Ausgabe,
- eine Bedingung,
- eine Schleife,
- eine Variable.

Schritt 1 - Idee

Beschreibe dein Projekt in drei Sätzen.

Schritt 2 - Algorithmus

Notiere den wichtigsten Ablauf zunächst als Schritte oder Ablaufdiagramm.

Schritt 3 - Umsetzung

Setze den Ablauf in Scratch um.

Schritt 4 - Testen

Führe mindestens drei unterschiedliche Tests durch.

Test	Aktion/Eingabe	Erwartetes Ergebnis	Tatsächliches Ergebnis	bestanden?
1				
2				
3				

Schritt 5 - Verbessern

Behebe gefundene Fehler und wiederhole die betroffenen Tests.

Schritt 6 - Erklären

Beantworte:

1. Wo findest du in deinem Projekt eine Sequenz?
2. Wo wird eine Bedingung verwendet?
3. Welche Schleife nutzt du und warum?
4. Was speichert deine Variable?
5. Wie lässt sich dein Projekt nach EVAS beschreiben?

Reflexion

Was war beim Programmieren schwierig? Wie hast du das Problem gelöst?

Ausblick

Jetzt hast du selbst einen Algorithmus geplant, programmiert und getestet. Als Nächstes prüfen wir, welche Grundlagen du sicher beherrschst.

14 - Kompetenzcheck

Kreuze zunächst selbst ein, wie sicher du dich fühlst.

Ich kann ...	sicher	teilweise	noch nicht
Binärzahlen lesen	☐	☐	☐
Dezimalzahlen in Binärzahlen umrechnen	☐	☐	☐
Binärzahlen addieren	☐	☐	☐
erklären, wie Texte digital gespeichert werden	☐	☐	☐
Pixel, RGB und Auflösung erklären	☐	☐	☐
Abtastrate und Bittiefe erklären	☐	☐	☐
EVAS auf ein Gerät anwenden	☐	☐	☐
einen Algorithmus beschreiben	☐	☐	☐
Sequenz, Bedingung und Schleife unterscheiden	☐	☐	☐
einfache Scratch-Programme lesen	☐	☐	☐
Variablen erklären	☐	☐	☐

Teil A - Binärzahlen

1. Wandle 19_{10} in eine Binärzahl um.
2. Wandle 101101_2 in eine Dezimalzahl um.
3. Berechne $1011_2 + 0101_2$.

Teil B - Digitale Information

4. Erkläre, warum ein Buchstabe im Computer als Zahl dargestellt werden kann.
5. Was ist ein Pixel?
6. Was bedeutet RGB?
7. Erkläre den Unterschied zwischen Abtastrate und Bittiefe.

Teil C - EVAS

Analysiere einen Fahrkartenautomaten nach Eingabe, Verarbeitung, Ausgabe und Speicherung.

Teil D - Algorithmen

8. Nenne drei Eigenschaften eines guten Algorithmus.
9. Ordne zu: Sequenz, Entscheidung oder Wiederholung.
 - „Wiederhole zehnmal: gehe einen Schritt.“
 - „Wenn die Ampel grün ist, gehe.“
 - „Nimm das Heft, öffne es und schreibe das Datum.“

Teil E - Scratch

10. Was ist ein Ereignis?
11. Wozu dient eine Variable?
12. Erkläre den Unterschied zwischen einer Bedingung und einer Schleife.

Auswertung

Markiere anschließend drei Themen, die du vor der Leistungskontrolle noch einmal üben möchtest.